Module End Assignment
“Analyzing E-Learning Platform Purchases using MySQL”

Introduction:

This project focuses on analyzing course and learner data using SQL. The objective is to extract meaningful insights that support business decision-making.

Objective:

An online learning platform sells various digital courses to learners across different countries. Each learner can purchase multiple courses, and each course belongs to a specific category. The management team wants to analyze purchase data to understand sales trends, learner behaviour, and popular course categories. The task is to use MySQL to design and query a relational database that helps derive meaningful insights from this e-learning purchase data.

DATASET DESCRIPTION:

1.Table: learners

Attributes:

- learner_id (Primary Key)
- full_name
- Country

2..Table: courses

Attributes:

- course_id (Primary Key)

• course_name

• category

• unit_price

3.Table: purchases

Attributes:

• purchase_id (Primary Key)

• learner_id (Foreign Key → learners.learner_id)

• course_id (Foreign Key → courses.course_id)

• Quantity

• Purchase_date

Tasks:

1. Create the database and schema. Populate the Schema:

- Create a Database for this project and
- Create all three tables in MySQL with appropriate **data types** and **relationships**.
- Insert **sample data** covering at least:
  - 4–5 learners
  - 4–5 courses (spread across multiple categories)
  - 6–8 purchase records

```sql
create database if not exists online_learning;
use online_learning;

create table learners(
learner_Id INT Primary Key auto_increment,
full_name varchar(100),
country varchar(50)
);

create table courses(
course_Id INT primary key auto_increment,
course_name varchar(100),
category varchar(100),
unit_price decimal(10,2)
);

create table purchase(
purchase_Id INT Primary key auto_increment,
learner_ID INT,
course_Id INT,
quantity INT,
purchase_date date,
foreign key (learner_Id) references learners(learner_Id),
foreign key (course_Id) references courses(course_Id)
);
```

```sql
INSERT INTO learners(full_name,country)
VALUES('Ajith Kumar','India'),('John Peter','USA'),
('Mark Putin','Russia'),('Maria','UK'),('Adam Smith','Germany'),
('Andrew San','Japan'),('Adam william','Australia'),('John wilson','Africa');


INSERT INTO courses(course_name,category,unit_price)
VALUES('Data Analyst','Intermediate',500.00),('Data Scientist','Beginner',700.00),
('Software Testing','Advanced',450.00),('Full Stack','Intermediate',600.00),
('Cloud Computing','Beginner',650.00),('cyber securiy','Advanced',900.00),
('Devops','Intermediate',1000.00),('Ai Basics','Intermediate',800.00);

INSERT INTO purchase(learner_id,course_id,quantity,purchase_date)
VALUES(1,1,3,'2026-04-01'),(2,2,4,'2026-04-02'),(3,3,1,'2026-04-03'),(4,7,16,'2026-04-20'),
(5,5,1,'2026-05-01'),(6,6,5,'2026-04-24'),(7,7,3,'2026-04-14'),(8,8,5,'2026-04-30');

select * from learners;
select*from courses;
select * from purchase;
```

2. Data Exploration Using Joins:

- Format currency values to 2 decimal places.
- Use aliases for column names (e.g., AS total_revenue).
- Sort results appropriately (e.g., highest total_spent first).

```sql
-- Data exploration using Joins
select l.learner_Id,l.full_name,
format(sum(p.quantity * c.unit_price) ,2) as total_spent
from learners l
join purchase p on l.learner_Id=p.learner_Id
join courses c on c.course_Id =p.course_Id
Group by l.learner_Id,l.full_name
order by total_spent desc;
```

Use SQL INNER JOIN, LEFT JOIN, and RIGHT JOIN to:

- Combine learner, course, and purchase data
- Display each learner's purchase details (course name, category, quantity, total amount, and purchase date).

```sql
-- Inner Join
select l.full_name as learner_name,c.course_name,
c.category,p.quantity,
ROUND(Sum(p.quantity * c.unit_price), 2) as total_amount,
p.purchase_date
from learners l
Inner Join purchase p on l.learner_Id = p.learner_Id
Inner Join courses c on p.course_Id = c.course_Id
Group by l.full_name,c.course_name,c.category,p.quantity,p.purchase_date
order by total_amount desc;

-- Left Join
select l.full_name as learner_name, c.course_name,
c.category,p.quantity,
ROUND(Sum(p.quantity * c.unit_price), 2) as total_amount,
p.purchase_date
from learners l
Left Join purchase p on l.learner_Id = p.learner_Id
Left Join courses c on p.course_Id = c.course_Id
Group by l.full_name,c.course_name,c.category,p.quantity,p.purchase_date
order by total_amount desc;

-- Right Join
select l.full_name as learner_name,c.course_name,
c.category,p.quantity,
ROUND(Sum(p.quantity * c.unit_price), 2) as total_amount,
p.purchase_date
from learners l
Right Join purchase p on l.learner_Id = p.learner_Id
Right  Join courses c on p.course_Id = c.course_Id
Group by l.full_name,c.course_name,c.category,p.quantity,p.purchase_date
order by total_amount desc;
```

Analytical Queries:

1. Display each learner's total spending (quantity × unit_price) along with their country.

```
select l.full_name as learner_name,l.country,
ROUND(Sum(p.quantity * c.unit_price) ,2) as total_spending
from learners l
Inner join purchase p on p.learner_id = l.learner_Id
Inner join courses c on c.course_Id = p.course_Id
Group by l.full_name,l.country
order by total_spending DESC;
```

2. Find the top 3 most purchased courses based on total quantity sold.

```
select c.course_Id,c.course_name,
(Sum(p.quantity)) as total_quantity_sold
from courses c
Inner join purchase p on c.course_Id = p.course_Id
Group by c.course_Id,c.course_name
order by total_quantity_sold desc
Limit 3;
```

3. Show each course category's total revenue and the number of unique learners who purchased from that category.

```
select c.category,
ROUND(Sum(p.quantity * c.unit_price) ,2) as total_revenue,
count(DISTINCT (p.learner_Id)) as Unique_learner
from courses c
Inner join purchase p on p.course_Id=c.course_Id
Group by c.category
order by unique_learner DESC;
```

4. List all learners who have purchased courses from more than one category.

```
select l.learner_Id,l.full_name as learner_name
from learners l
join purchase p on p.learner_Id = l.learner_Id
 Join courses c  on c.course_Id = p.course_Id
Group by l.learner_Id,l.full_name
Having count(Distinct c.category>1);
```

5.Identify courses that have not been purchased at all.

```
select c.course_Id,c.course_name,c.category
from courses c
left join purchase p on p.course_Id = c.course_Id
where p.course_Id IS NULL;
```

TYPES OF ANALYSIS PERFORMED:

➢ Descriptive Analysis:

Descriptive analysis was performed using SQL functions such as COUNT(), SUM(), and GROUP BY to summarize the dataset and understand overall performance of courses and learners.

➢ Diagnostic Analysis:

Diagnostic analysis was carried out by joining multiple tables to identify patterns and reasons behind course performance, such as low sales in certain categories and inactive courses.

➢ Predictive Analysis:

Based on historical data trends, it is observed that courses with higher purchase counts are likely to perform well in the future. Similarly, active learners are more likely to continue purchasing courses.

➢ Prescriptive Analysis:

The following recommendations are suggested:

- Promote courses with zero or low purchases
- Focus marketing efforts on high-performing categories
- Provide offers to inactive learners
- Bundle popular courses to increase sales.

KEY INSIGHTS:

• Certain categories have significantly higher number of courses

• Top-selling courses contribute majorly to total sales

• Some courses have no purchases and require attention

• A few learners purchase across multiple categories, indicating high engagement

CONCLUSION:

This project helped in understanding and applying SQL concepts such as joins, aggregation, and filtering. It also improved analytical thinking by interpreting data and deriving meaningful insights. The analysis provided valuable recommendations to improve course performance and learner engagement.